

CO 2 ASSESSMENT TOOL 1
COURSE NAME: Computer Vision with OpenCV
COURSE CODE: DSA02

NAME: N Yugesh
Reg No: 192424329

Question 1: Salt-and-Pepper Noise Removal Before Edge Detection

***Scenario:** A 1024×1024 grayscale surveillance image is corrupted by salt-and-pepper noise with density 0.05. The image must be preprocessed before edge detection, which is sensitive to abrupt intensity changes.*

a. Most suitable filtering technique

The median filter is the most suitable choice. Salt-and-pepper noise replaces pixel values with extreme intensities (0 or 255) rather than adding a small offset, so it behaves as an outlier in any local neighborhood. The median filter replaces each pixel with the median of its neighborhood, which is a non-linear, rank-based operation that discards outliers entirely rather than blending them into the result. This removes the corrupted pixels while keeping edges — which appear as a rank-order step in the neighborhood, not as an outlier — largely intact.

b. Why the median filter outperforms an averaging (mean) filter

An averaging filter is linear: it sums all neighborhood values (including the extreme 0/255 outliers) and divides by the neighborhood size. Because salt-and-pepper values are far from the true intensity, they pull the average strongly toward themselves, so noise is only diluted, not removed, and edges are blurred in the process since averaging smooths across the discontinuity. The median filter, in contrast, ranks the neighborhood values and selects the middle one. As long as noisy pixels are a minority within the window, they are pushed to the extremes of the sorted list and never selected as the median, so the output equals a genuine (noise-free) intensity. This also preserves sharp transitions better, since the median tracks the dominant value on either side of an edge instead of blending them.

c. Probability that a 3×3 median filter corrects a noisy center pixel

Assumptions: noise density $p = 0.05$ is applied independently to each pixel (including the center and its 8 neighbors), border effects are ignored (assume the pixel is not on the image boundary, or that symmetric/replicate padding is used), and the median filter "corrects" the center pixel whenever a strict majority (at least 5 of the 9 pixels in the window) are noise-free — in that case the sorted median must come from a clean pixel.

Let X be the number of noisy pixels among the 9 window pixels; $X \sim \text{Binomial}(n = 9, p = 0.05)$. The filter corrects the center pixel when $X \leq 4$:

$$P(\text{correct}) = \sum_{k=0}^4 C(9,k) \cdot (0.05)^k \cdot (0.95)^{9-k} \approx 0.99997 \quad (\approx 99.997\%)$$

So under these assumptions, a 3×3 median filter corrects an individual noisy center pixel with very high probability (essentially certain) at this noise density, because 0.05 makes it extremely unlikely that 5 or more of the 9 window pixels are simultaneously corrupted.

d. Total number of noisy pixels in the original image

$$\text{Total pixels} = 1024 \times 1024 = 1,048,576$$

$$\text{Noisy pixels} \approx \text{density} \times \text{total pixels} = 0.05 \times 1,048,576 \approx 52,429 \text{ pixels}$$

e. Expected number of noisy pixels remaining after median filtering

Using the per-pixel correction probability from part (c), the probability that the filter fails to correct a given noisy pixel is:

$$P(\text{fail}) = 1 - P(\text{correct}) \approx 1 - 0.99997 = 0.00003$$

Expected remaining noisy pixels $\approx$ (noisy pixels) $\times$ P(fail):

$$\approx 52,429 \times 0.00003 \approx 1\text{--}2 \text{ pixels}$$

So median filtering is expected to leave only a handful of residual noisy pixels (roughly 1–2 out of the original ~52,429) — essentially the small fraction of locations where multiple noisy pixels happened to cluster in the same 3×3 window.

f. If the noise were Gaussian instead of salt-and-pepper

Gaussian noise adds a small random offset drawn from a normal distribution to every pixel (not just a random subset), so there are no extreme outlier values to reject — a rank-based filter has nothing to "throw out." The preprocessing strategy would shift from an outlier-rejection filter to a smoothing/statistical filter: a Gaussian blur, a mean filter, or (for better edge preservation) a bilateral filter or Wiener filter would be preferred over a median filter. These linear/adaptive smoothing filters reduce the variance of the additive noise by local averaging while a bilateral or non-local-means filter additionally preserves edges by limiting averaging to photometrically similar pixels. A median filter would still work to some degree but is not the optimal choice for Gaussian noise, since it is specifically designed for impulsive (outlier-type) noise.

Question 2: Discretization of a Continuous Image

Scenario: An analog grayscale image has a smooth sinusoidal intensity variation along x and is constant along y , defined over a spatial range of 0 to 255 in both directions. It must be converted to a digital image.

a. Converting the continuous image to digital form

Digitization requires two independent steps. Sampling discretizes the spatial coordinates: the continuous intensity function $f(x, y)$ is evaluated only at a regular grid of points ($x = m \cdot \Delta x$, $y = n \cdot \Delta y$), converting the continuous domain into a finite array of pixels. Quantization discretizes the intensity axis: the continuous range of intensity values at each sampled point is mapped to one of a finite set of discrete levels (e.g., 256 levels for 8-bit representation). Applying both operations

together turns the continuous image $f(x, y)$ into a digital image $f[m, n]$ with integer coordinates and integer (quantized) intensity values, suitable for storage and computer processing.

b. Spatial variation pattern in the x-direction

The intensity varies as a one-dimensional sinusoid along x — i.e., $f(x, y) = A \cdot \sin(2\pi f_0 x + \phi) + B$, independent of y . Because the pattern does not change with y , the image appears as a series of vertical bright/dark bands (a grating pattern) running parallel to the y -axis, with intensity oscillating smoothly and periodically along the x -axis.

c. Minimum sampling requirement to avoid aliasing

By the Nyquist–Shannon sampling theorem, the sampling frequency along x must be at least twice the highest spatial frequency f_0 present in the signal:

$$f_s \geq 2 \cdot f_0 \quad (\text{i.e., at least 2 samples must be taken per cycle of the sinusoid})$$

Equivalently, the sampling interval Δx must satisfy $\Delta x \leq 1/(2f_0)$. If the sinusoid completes a known number of full cycles N over the 0–255 range, the minimum number of samples required across that range is $2N$; in general, without a specific f_0 given, the qualitative requirement is that the sampling rate must be at least twice the sinusoid's frequency — sampling any slower than this violates Nyquist and introduces aliasing.

d. Effect of sampling below the required rate

If the image is sampled below the Nyquist rate, aliasing occurs: the high-frequency sinusoidal pattern is misrepresented as a lower-frequency pattern in the sampled image. This produces visible artifacts such as moiré patterns, jagged or false banding, and a fundamentally incorrect reconstruction of the original intensity variation — the digital image will show a spurious slower oscillation that was never present in the analog signal.

e. Suitable 8-bit quantization scheme

Since the image is grayscale with a smooth, continuous sinusoidal intensity range, a uniform (linear) 8-bit quantization scheme is appropriate: the continuous intensity range is divided into 256 equal-width levels (0–255), with each sampled value rounded to the nearest level. Because the sinusoid varies smoothly with no abrupt jumps, uniform quantization introduces evenly distributed, low-magnitude quantization error and needs no special non-linear (e.g., companding) treatment.

f. Effect of quantization on image quality, storage, and accuracy

Increasing the number of quantization levels (e.g., moving from 8-bit to a higher bit depth) improves intensity accuracy and produces smoother-looking gradients, reducing visible artifacts such as false contouring/banding in smooth regions like this sinusoid. However, it also increases storage size proportionally, since more bits are required per pixel. Conversely, coarser quantization (fewer levels) reduces storage but increases quantization error, causing visible banding in smooth intensity transitions and a loss of fine intensity detail. Choosing the quantization level is therefore a trade-off between image fidelity and storage/bandwidth cost.

Question 3: Edge Detection in Thin Line Images

Scenario: A 512×512 grayscale image contains very thin (1-pixel-thick) vertical and horizontal white lines on a dark background, used in document/engineering-drawing/circuit analysis, and lines must be detected without losing thin edge information.

a. Comparing Sobel, Prewitt, and Canny for 1-pixel-thick lines

- Sobel: A first-order gradient operator that weights the center row/column more heavily, giving slightly better noise suppression than Prewitt, but it still produces a thick gradient-magnitude response (several pixels wide) around thin lines, requiring separate thresholding/thinning.
- Prewitt: Very similar to Sobel but with uniform weighting; it is more sensitive to noise and gives a comparatively less accurate gradient estimate, though computationally simpler. It also yields multi-pixel-wide edge responses.
- Canny: The most suitable of the three, since it combines Gaussian smoothing, gradient computation, non-maximum suppression, and hysteresis thresholding. Non-maximum suppression specifically thins the gradient response down to a single-pixel-wide edge, and hysteresis thresholding helps retain faint but genuine thin-line edges while suppressing noise — making Canny best-suited for preserving 1-pixel-thick line structures.

b. Sobel gradient response near a horizontal edge at (256, 256)

Assume a representative 3×3 neighborhood centered at (256, 256) straddling a horizontal white line (intensity 255) on a dark background (intensity 0), i.e. the row above/below the line is 0 and the line row itself is 255:

$$\text{Neighborhood: } \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} 255 & 255 & 255 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

$$G_x \text{ kernel} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y \text{ kernel} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Since each row is constant, G_x (horizontal gradient) = 0, because the kernel differences columns of identical values. G_y (vertical gradient), which detects horizontal edges, gives:

$$G_y = (-1 \cdot 0 - 2 \cdot 0 - 1 \cdot 0) + (0) + (1 \cdot 0 + 2 \cdot 0 + 1 \cdot 0) \rightarrow \text{using the line row: } G_y = [(-1)(0) + (-2)(0) + (-1)(0)] + [0] + [(1)(0) + (2)(0) + (1)(0)] = 0$$

Recomputing properly with the line centered at the middle row (255) and 0 above/below: $G_y = (\text{top row} \cdot [-1, -2, -1]) + (\text{bottom row} \cdot [1, 2, 1]) = (0 \times -1 + 0 \times -2 + 0 \times -1) + (0 \times 1 + 0 \times 2 + 0 \times 1) = 0$, since both neighboring rows are 0. The strong gradient response instead occurs one row above and one row below the line center, where $G_y = \pm(255 + 2 \cdot 255 + 255) = \pm 1020$, giving a gradient magnitude $|G| = \sqrt{G_x^2 + G_y^2} = 1020$ at those adjacent rows. This confirms the edge is correctly localized at the top and bottom boundary of the 1-pixel line rather than at its exact center.

c. Role of gradient magnitude and direction

Gradient magnitude, $|G| = \sqrt{G_x^2 + G_y^2}$, indicates the strength (contrast) of an edge — large magnitude means a sharp intensity transition, while small magnitude indicates a weak or negligible edge, which is used for thresholding to decide whether a pixel is an edge at all. Gradient direction, $\theta = \text{atan2}(G_y, G_x)$, indicates the orientation of the edge (perpendicular to the direction of steepest intensity change), which is used to determine whether an edge is horizontal, vertical, or diagonal, and is essential for non-maximum suppression, since suppression is performed along the gradient direction to thin the edge to one pixel.

d. Role of Gaussian smoothing in Canny edge detection

Gaussian smoothing is applied as the first step in the Canny pipeline to suppress high-frequency noise before gradient computation. Because differentiation (used to compute gradients) amplifies noise, smoothing first prevents spurious gradient peaks from being detected as false edges, resulting in cleaner, more reliable edge maps.

e. Effect of Gaussian smoothing on thin-line localization

While Gaussian smoothing reduces noise, it also blurs the image, and for 1-pixel-thick lines this can be especially harmful: a large smoothing kernel (large σ) can spread the line's intensity across several neighboring pixels, reducing its peak contrast, shifting or widening the apparent edge location, and in extreme cases merging closely spaced thin lines or nearly erasing very faint thin lines altogether. This creates a trade-off — a larger σ gives better noise suppression but poorer localization/preservation of thin structures, so a small σ is generally preferred when thin lines must be preserved.

f. Effect of halving the Canny threshold

Lowering the high/low threshold pair reduces the bar for a pixel to be classified as a strong or weak edge. As a result: more edges overall would be detected (increased sensitivity), more weak edges would be retained and, through hysteresis, connected into the final edge map (since weak edges adjacent to strong edges are kept), and false edges (from residual noise or minor texture) would also increase, since the detector becomes less selective about what counts as a genuine intensity transition. In short, recall improves (fewer real edges missed) at the cost of precision (more spurious/noisy edges appear).

Question 4: Feature Extraction for Handwritten Digit Recognition

Scenario: Handwritten digits vary in size, orientation, stroke thickness, and writing style, so robust feature extraction is required before classification.

a. Working principles of SIFT, SURF, and ORB

- SIFT (Scale-Invariant Feature Transform): Builds a scale-space using a Difference-of-Gaussians pyramid to detect stable keypoints across scales, assigns each keypoint a dominant orientation for rotation invariance, and describes the local neighborhood using a 128-dimensional histogram of gradient orientations.
- SURF (Speeded-Up Robust Features): Approximates the Gaussian-based scale space using box filters and integral images for speed, detects keypoints via a Hessian-matrix-based determinant, and builds a 64-dimensional descriptor from Haar-wavelet responses — designed to be faster than SIFT while remaining scale/rotation invariant.
- ORB (Oriented FAST and Rotated BRIEF): Combines the FAST keypoint detector with an orientation component (for rotation invariance) and the BRIEF binary descriptor (rotated according to keypoint orientation), producing a compact binary descriptor that can be compared using fast Hamming distance instead of Euclidean distance.

b. Comparison: scale invariance, rotation invariance, computational cost, real-time suitability

Method	Scale Invariance	Rotation Invariance	Computational Cost	Real-time Suitability
SIFT	Excellent	Excellent	High	Low (slow)
SURF	Good	Good	Medium	Moderate
ORB	Moderate	Good	Low	High (fast)

c. Reliability under a 45° rotation

All three methods — SIFT, SURF, and ORB — explicitly compute a dominant keypoint orientation and normalize their descriptors relative to it, so all three are designed to remain reliable under a 45° rotation. SIFT and SURF, having stronger orientation-normalization and richer gradient-histogram descriptors, tend to be the most robust to rotation; ORB's rotated BRIEF also handles 45° rotation well and is a good practical choice when speed matters, though it is generally considered slightly less robust than SIFT/SURF for extreme viewpoint changes.

d. Total keypoints detected across 50 images

Total keypoints = average keypoints per image × number of images = 120 × 50 = 6,000 keypoints

e. Dimensionality reduction technique retaining 95% of information

Principal Component Analysis (PCA) is a suitable technique: it projects the high-dimensional feature vectors (e.g., 128-D SIFT descriptors) onto a smaller set of orthogonal principal components ranked by explained variance, and the number of components is chosen such that their cumulative explained variance ratio reaches 95%, discarding the remaining low-variance (largely redundant/noisy) components.

f. How dimensionality reduction improves classification efficiency

Reducing feature dimensionality decreases the computational cost and memory required for storing and comparing feature vectors, speeds up both training and inference in the classifier, and helps mitigate the curse of dimensionality (where classifiers need exponentially more data to generalize well as dimensionality grows). By removing redundant or highly correlated dimensions while retaining 95% of the meaningful variance, it also reduces overfitting risk and can improve generalization, since the classifier is trained on the most informative, decorrelated feature components.

Question 5: Morphological Preprocessing for Character Recognition

Scenario: A 256×256 binary image of scanned handwritten characters contains small white noise dots in the background and small black gaps inside character strokes, which must be cleaned using morphological preprocessing before recognition.

a. Operation to remove small white noise dots

The Opening operation (erosion followed by dilation) is suitable. Erosion first shrinks and eliminates small isolated white noise dots (since they are smaller than the structuring element and get fully eroded away), and the subsequent dilation restores the size of the remaining, larger character strokes without bringing back the removed noise.

b. Operation to fill small gaps/holes inside strokes

The Closing operation (dilation followed by erosion) is suitable. Dilation first expands the character strokes enough to bridge and fill small internal black gaps/holes, and the subsequent erosion shrinks the strokes back toward their original size while keeping the gaps filled in.

c. Difference between opening and closing

Opening = erosion followed by dilation. It removes small bright (foreground) protrusions and isolated small objects while roughly preserving the size/shape of larger objects — it "opens up" narrow connections and eliminates small bright noise. Closing = dilation followed by erosion. It fills small dark gaps/holes and connects nearby bright regions separated by thin gaps — it "closes up" small breaks. In short, opening removes small bright noise, while closing fills small dark holes/gaps; they are morphological duals of each other.

d. Effect of a 3×3 square structuring element on thin strokes

Because handwritten character strokes are often only 1–2 pixels wide, a 3×3 structuring element is comparable to or larger than the stroke width itself. During the erosion step (in opening), thin strokes risk being significantly thinned, broken into disconnected segments, or entirely removed if they are narrower than the structuring element. The subsequent dilation in opening may not fully restore fine details that were lost. This means small structuring elements must be chosen carefully relative to expected stroke width to avoid damaging genuine character strokes while still removing noise.

e. Risk of using a large structuring element

A large structuring element increases the risk of over-erosion/over-dilation: legitimate thin strokes can be completely erased (in opening) or adjacent characters/strokes can be merged together (in closing), distorting character shape, connecting separate letters, or losing fine details like serifs or dots (e.g., on 'i' or 'j'). This degrades the accuracy of downstream feature extraction and recognition, since the morphological cleanup ends up removing meaningful signal along with the noise.

f. Complete preprocessing pipeline for handwritten character recognition

- 1. Image Acquisition: Scan/capture the handwritten document image.
- 2. Grayscale Conversion: Convert the color/scanned image to a single-channel grayscale image.
- 3. Binarization/Thresholding: Apply a global or adaptive threshold (e.g., Otsu's method) to convert the grayscale image into a binary (black/white) image separating strokes from background.
- 4. Noise Removal (Opening): Apply a small structuring element opening to remove small white noise dots from the background.
- 5. Gap Filling (Closing): Apply a small structuring element closing to fill small internal gaps/holes inside the strokes.
- 6. Thinning/Skeletonization: Reduce strokes to a 1-pixel-wide skeleton to normalize stroke thickness variation across writers.
- 7. Normalization: Resize/center each character in a fixed-size bounding box to standardize scale and position.

- 8. Feature Extraction: Extract descriptive features (e.g., HOG, contour/shape descriptors, or CNN-based features) from the cleaned, normalized character image.
- 9. Classification: Feed the extracted features into a classifier (e.g., SVM, k-NN, or a neural network) to recognize the character.